Manual técnico

Instalación, configuración y mantención

Sistema de Gestión de Taller, Inventario y Ventas — Lubri-Express

Este manual es para quien instala y mantiene el sistema en el equipo del taller. El uso diario está en el *Manual de usuario*.

Índice

1. Qué es y qué necesita **2**

2. Instalación en el equipo del taller **2**
- 2.1. 1. PostgreSQL . . . 2
- 2.2. 2. La aplicación . . . 2
- 2.3. 3. El archivo .env . . . 3
- 2.4. 4. El primer administrador . . . 3

3. Migrar los datos del sistema anterior **3**
- 3.1. Cómo leer el informe . . . 3
- 3.2. Después de migrar: fijar los mínimos . . . 4

4. Importar las planillas de cobrar y pagar **4**
- 4.1. Qué dice el informe . . . 4
- 4.2. Aplicar y deshacer . . . 4

5. Respaldos **5**
- 5.1. Respaldar . . . 5
- 5.2. Restaurar . . . 5
- 5.3. Dejarlo automático . . . 5
- 5.4. Una advertencia sobre las versiones . . . 6

6. Empaquetar la aplicación **6**

7. Correr desde el código fuente **6**

8. Versiones **6**

9. Actualizar a una versión nueva **7**
- 9.1. La versión 1.1.3: el cliente de cada orden . . . 7
- 9.2. La versión 1.2.0: exportar el inventario . . . 8

10. Problemas frecuentes **8**

11. Verificación después de instalar **8**

1 Qué es y qué necesita

Aplicación de escritorio en Python (PySide6) contra una base de datos PostgreSQL, ejecutándose **íntegramente en el equipo del taller**: no depende de internet y los datos no salen de ahí.

Componente	Versión
Windows	10 o superior (64 bits)
PostgreSQL	16 o superior
Python	3.10 o superior (**no** hace falta en el equipo del taller)
Espacio en disco	1 GB para la aplicación, más la base y los respaldos

El sistema se puede ejecutar de dos maneras: desde el **ejecutable** empaquetado (lo normal en el taller) o desde el **código fuente** (para desarrollo). La carpeta empaquetada trae, además de la aplicación, las cuatro herramientas de mantención que hacen falta en el equipo, de modo que ahí **no se necesita Python instalado**:

Ejecutable	Para qué
`lubriexpress.exe`	La aplicación
`actualizar.exe`	Lleva la base a la versión nueva del esquema
`respaldar.exe`	El volcado diario, para la tarea programada
`restaurar.exe`	Pone de vuelta un respaldo: el camino de regreso
`crear_usuario.exe`	El primer administrador de una instalación nueva

La migración del sistema anterior queda fuera a propósito: se hace una sola vez, desde el código fuente y con las planillas, que no viven en el repositorio.

2 Instalación en el equipo del taller

2.1 1. PostgreSQL

Instalar PostgreSQL 16 desde `https://www.postgresql.org/download/windows/`. Durante la instalación se pide una contraseña para el usuario `postgres`: hay que anotarla, porque va en el archivo de configuración. Dejar marcada la instalación de las *Command Line Tools*: de ahí salen `pg_dump` y `psql`, que usan los respaldos.

Crear la base y cargar el esquema, desde el *SQL Shell (psql)* o desde `cmd`:

```
createdb -U postgres lubriexpress
psql -U postgres -d lubriexpress -v ON_ERROR_STOP=1 -f schema_lubriexpress.sql
```

El archivo `schema_lubriexpress.sql` está en `database/` del código fuente. Crea las tablas, las restricciones, los triggers de Kardex y la vista de stock crítico.

No hace falta configurar el huso horario del servidor: la aplicación lo fija en cada conexión. Importa más de lo que parece, porque la hora de cada venta, abono y movimiento la pone el servidor; con el servidor en UTC, una venta de las nueve de la noche quedaría fechada al día siguiente y se caería del cierre de caja y del reporte del día.

2.2 2. La aplicación

Copiar la carpeta `lubriexpress` (la que produce el empaquetado, sección 6) al equipo, por ejemplo en `C:\Lubri-Express`. Dentro van los cinco ejecutables y todo lo que necesitan.

2.3 3. El archivo .env

Junto al ejecutable debe quedar un archivo llamado `.env` con la dirección de la base:

```
DATABASE_URL=postgresql+psycopg2://postgres:LA_CONTRASENA@localhost:5432/lubriexpress
```

Sin ese archivo la aplicación no arranca, y lo dice con un cartel. Es a propósito: más vale que no abra a que se conecte en silencio a una base que no es la que se creía.

Ese mismo `.env` lo leen las cuatro herramientas de consola, y la carpeta `respaldos\` también se crea ahí, **junto al ejecutable**. Adentro del paquete no sirve de nada: esas rutas apuntan a una carpeta temporal que Windows borra al cerrar el programa.

2.4 4. El primer administrador

Los usuarios se administran desde la aplicación, pero el primero hay que crearlo por consola:

```
crear_usuario.exe
```

Pide nombre, usuario, rol y contraseña. Elegir rol `ADMINISTRADOR`. De ahí en adelante, todos los demás se crean desde la pestaña **Usuarios**.

Si alguna vez se pierde la contraseña del administrador, el mismo script la reemplaza: al escribir un usuario que ya existe, lo actualiza.

3 Migrar los datos del sistema anterior

Se hace una sola vez, antes de empezar a operar.

```
.venv\Scripts\python scripts\migrar_sistema_antiguo.py "C:\ruta\a\las\planillas" --
    detalle
```

La carpeta debe tener las cinco planillas: `tabla clientes.xlsx`, `tabla vehiculo.xlsx`, `tabla productos.xlsx`, `tabla servicios.xlsx` y `tabla ordenes.xlsx`.

Todo entra en **una sola transacción**: o queda todo o no queda nada. El script se niega a correr dos veces (si ya existe el usuario `sistema_antiguo`, no hace nada), así que no hay forma de duplicar los datos por error.

3.1 Cómo leer el informe

Al terminar imprime cuánto cargó y, agrupado por motivo, cuánto descartó. Lo que hay que mirar:

- **Productos sin costo.** El sistema anterior tenía servicios anotados como productos ("Mano Obra", "1/4 Hora taller") con stock ficticio de miles de unidades. No se cargan. Con `-detalle` sale la lista: conviene revisarla y crear a mano los que sean servicios de verdad.
- **Stock negativo o fraccionario.** Se cargan con stock cero y hay que contarlos en la repisa (ajuste por recuento).
- **Patentes inválidas.** Autos extranjeros o datos mal escritos. Sus órdenes tampoco entran: no se les inventa un vehículo.
- **Categorías unificadas.** Las categorías venían escritas de cualquier forma: 131 escrituras que se consolidan en unas cuarenta categorías limpias. Se junta lo que es la misma palabra (mayúsculas, tildes, conectores, plurales) y erratas evidentes mediante equivalencias auditadas, conservando la escritura más usada. Con `-detalle` sale qué se unificó con qué.

- **Categorías parecidas.** Las palabras dudosas no se fusionan a ciegas: *Aceite moto* no es un *Aceite motor* mal escrito, y comparar letras no sabe la diferencia. Casos como ese salen listados con su cuenta ("Aceite moto (4) ˜ ¿Aceite motor (144)?") para que cualquier decisión quede en manos del taller.
- **Homónimos fundidos.** El sistema anterior identificaba al cliente solo por su nombre, así que dos clientes con el mismo nombre quedan como uno. Si aparece uno, se separa a mano.

Al final verifica que el stock cuadre: lo que decían las planillas, lo que quedó en el Kardex y lo que quedó en los productos tienen que ser el mismo número. Si no cuadra, no confirma nada.

3.2 Después de migrar: fijar los mínimos

Los productos migrados quedan con stock mínimo cero, así que todo lo que esté en cero aparece como crítico. Se corrige en Inventario con **Mínimo por categoría**, una categoría a la vez.

4 Importar las planillas de cobrar y pagar

Las planillas de Excel con que se llevaban las cuentas por cobrar y por pagar antes de la pestaña **Finanzas** se pueden cargar con un script. Es una herramienta de una sola vez, como la migración: corre desde el código fuente y las planillas no viven en el repositorio, así que se le pasa su ruta.

```
.venv\Scripts\python scripts\importar_planillas.py --cobrar "C:\ruta\X_COBRAR.xlsx"
    --pagar "C:\ruta\X_PAGAR.xlsx"
```

Así, **no escribe nada**: muestra un informe de lo que haría. Se revisa, y si está bien se repite el mismo comando con `-aplicar`. Se puede pasar solo una de las dos planillas.

4.1 Qué dice el informe

- **Por cobrar**: cuántas cuentas se crearían, cuántas pagadas y cuáles quedan por cobrar. Las filas **sin factura** no se importan: una cuenta sin orden detrás exige factura, así que esas órdenes siguen en la planilla hasta que se emitan. El informe las lista con el motivo.
- **Por pagar**: las facturas distintas, cuáles quedan con saldo y las repetidas que se descartaron (una misma factura aparece en varias hojas). Las filas de cuotas, que no traen proveedor ni número, se cuentan como pagos de la factura de arriba.
- **Proveedores**: la planilla escribe cada proveedor de varias maneras ("Lub Chile", "Lubchile Ltda." ...). El script las unifica y el informe muestra cada grupo (nombre en la planilla → nombre que se usará). Hay que mirarlo: los casos dudosos se dejan separados.
- **Avisos**: lo que se importa pero conviene revisar, como una factura marcada "Pagado" con un abono menor al monto (se toma como pagada completa) o un pago con fecha anterior a la compra.

4.2 Aplicar y deshacer

Con `-aplicar`, antes de escribir se hace un **respaldo**, y si falla no se toca nada; todo entra en **una sola transacción**. Lo importado queda a nombre de un usuario inactivo (`importacion_planillas`) y con la observación "Importada de planilla" más la hoja y la fila de origen. Volver a correrlo **no duplica**: lo que ya existe, por número de factura, se salta.

```
.venv\Scripts\python scripts\importar_planillas.py --deshacer --aplicar
```

`-deshacer` saca todo lo importado, con sus pagos, y también pide `-aplicar` para confirmar. Los proveedores creados se dejan, porque no llevan marca y pueden tener facturas ingresadas a mano.

5 Respaldos

5.1 Respaldar

```
respaldar.exe
```

Deja un archivo comprimido en `respaldos\lubriexpress-AAAAMMDD-HHMM.sql.gz`, conserva los últimos 30 y borra los más viejos. Si en el `.env` está configurada la carpeta de OneDrive, deja además una copia ahí:

```
RESPALDO_ONEDRIVE=C:\Users\TU_USUARIO\OneDrive\Respaldos Lubri-Express
```

El cliente de OneDrive sube solo lo que cae en su carpeta, así que con copiar el archivo basta. Si la carpeta no existe, el script avisa y el respaldo local igual queda hecho.

`respaldar.exe -listar` muestra lo que hay guardado.

Si PostgreSQL no está instalado en el equipo sino dentro de un contenedor de Docker, se le dice cuál en el mismo `.env`:

```
RESPALDO_CONTENEDOR=lubriexpress-db
```

5.2 Restaurar

```
restaurar.exe respaldos\lubriexpress-20260916-2030.sql.gz
```

Pisa la base entera. Por eso pide escribir el nombre de la base para confirmar. Conviene probar una restauración al menos una vez, contra una base de prueba, antes de necesitarla de verdad.

5.3 Dejarlo automático

En el Programador de tareas de Windows, o con este comando en `cmd` como administrador (ajustando las rutas):

```
schtasks /create /tn "Respaldo Lubri-Express" /tr "\"C:\Lubri-Express\lubriexpress\
    respaldar.exe\"" /sc daily /st 20:00 /f
```

Las comillas dobles escapadas son a propósito. Si la ruta tiene un espacio (por ejemplo `C:\Users\Lubri Express\...`), Windows corta el comando en el espacio y busca un programa que no existe: la tarea se crea sin quejarse, pero cada ejecución termina con el resultado `-2147024894` (`0x80070002`, archivo no encontrado) y no se hace ningún respaldo. Con `schtasks /query /tn "Respaldo Lubri-Express" /v /fo list` se ve el *Último resultado*; tiene que decir `0`. El `/f` reemplaza una tarea que ya existía.

A las 20:00, después del cierre. Conviene revisar cada tanto que los archivos estén apareciendo, tanto en `respaldos\` como en OneDrive: una tarea programada que dejó de correr no avisa, y eso se descubre el día que hace falta el respaldo.

5.4 Una advertencia sobre las versiones

El respaldo lo hace `pg_dump`, y **tiene que ser de la misma versión mayor que el servidor**: uno más nuevo escribe instrucciones que el servidor viejo no entiende al restaurar, y eso se descubre el peor día posible. El script lo verifica antes de escribir nada y se niega si no calzan. Si aparece ese mensaje, hay que instalar las herramientas cliente de la versión que corresponda.

6 Empaquetar la aplicación

El ejecutable **se construye en Windows**: PyInstaller no cruza de un sistema operativo a otro. Desde el código fuente, con el entorno virtual creado:

```
.venv\Scripts\pyinstaller.exe --noconfirm lubriexpress.spec
```

Queda en `dist\lubriexpress\`, con los cinco ejecutables. Esa carpeta completa es lo que se copia al equipo del taller; el `.env` va adentro, al lado de los `.exe`.

El archivo `lubriexpress.spec` incluye `qtbase_es.qm`, que es lo que hace que los botones de los diálogos digan “Guardar” y “Cancelar” en vez de “Save” y “Cancel”.

7 Correr desde el código fuente

Necesario para la migración, los respaldos y el empaquetado.

```
git clone https://github.com/CristobalEsp01/sist_lubriexpress.git
cd sist_lubriexpress
python -m venv .venv
.venv\Scripts\pip install -r requirements-dev.txt
copy .env.example .env
.venv\Scripts\python main.py
```

La batería de pruebas corre contra una base PostgreSQL real:

```
.venv\Scripts\python -m pytest
```

Si la base no está levantada, las pruebas que la necesitan se saltan solas en vez de fallar.

8 Versiones

El sistema numera sus versiones `MAYOR.MENOR.PARCHE`. El número vive en un solo lugar, `src/version.py`, y de ahí sale el que muestran el inicio de sesión, el título de la ventana y la barra de estado.

- **PARCHE** (1.1.**1**): corrige errores, sin funciones nuevas.
- **MENOR** (1.**2**.0): agrega una función o un módulo sin romper lo anterior.
- **MAYOR** (**2**.0.0): un cambio grande, por ejemplo una reestructuración de la base o de cómo se usa el sistema.

El número se cambia **a mano** al cerrar una versión. Cada una lleva su encabezado en las *Notas del parche*, un commit y una etiqueta de git (`v1.1.1`), y se empaqueta después de todo eso, para que los ejecutables lleven el número correcto. Si la versión cambia el esquema de la base, hay que correr `actualizar.exe` en el equipo del taller antes de abrirla.

9 Actualizar a una versión nueva

Con la base ya en uso, **no se recrea**: se actualiza. El paso 2 es el que importa, y hace todo solo.

1. Traer el código nuevo, volver a empaquetar (sección 6) y reemplazar la carpeta del ejecutable, **conservando el** `.env`.
2. Correr el actualizador:

```
actualizar.exe --simular
actualizar.exe
```

Con `-simular` dice qué haría y no escribe nada: sirve para mirar la lista antes de aceptarla. Sin la opción, hace esto en orden:

1. **Respalda.** Si el respaldo falla, no toca el esquema.
2. **Aplica los cambios de esquema**, todos en una sola transacción: uno que falle no deja la base a medio camino.
3. **Pregunta** por los cambios que tocan datos y no estructura — por ejemplo rellenar la ubicación de bodega que el sistema anterior había dejado escrita en la descripción del producto, o darle su tipo a los productos de categoría genérica ("FILTRO", "ACEITES") cuando el nombre lo dice— y muestra la lista antes de escribir.
4. **Verifica**: compara el número de filas de cada tabla antes y después. Un cambio de esquema no borra filas; si alguna cifra se movió, lo dice y hay que restaurar el respaldo.

Cada paso mira la base antes de actuar y se salta lo que ya está, así que **correrlo dos veces no hace daño**. Si algo no cuadra, el camino de vuelta es el respaldo que el propio actualizador acaba de dejar:

```
restaurar.exe respaldos\lubriexpress-AAAAMMDD-HHMM.sql.gz
```

El detalle de cada cambio de esquema está en `docs/base-de-datos.md`.

9.1 La versión 1.1.3: el cliente de cada orden

Hasta la 1.1.2 una orden no sabía a qué cliente pertenecía: lo leía del vehículo, y el vehículo tiene un solo dueño. Si el auto pasaba a otra persona, todas sus órdenes anteriores pasaban con él, con su PDF y lo que se debía. Desde la 1.1.3 la orden guarda su propio cliente, en `ordenes.cliente_id`:

- Lo copia el trigger `fn_congelar_cliente` al crear la orden, desde el dueño del vehículo ese día, y no se mueve más. La aplicación no lo escribe.
- El historial y su búsqueda, las órdenes abiertas, el detalle, el PDF, Finanzas y el reporte de descuentos leen el cliente de la orden, no el del vehículo.
- El **traspaso** de un auto, desde la ficha del vehículo, cambia solo el dueño actual (`vehiculos.cliente_id`). La ficha, el último servicio y las órdenes anteriores quedan como estaban.

Al actualizar, el paso `ordenes.cliente_id` de `actualizar.exe` agrega la columna y rellena las órdenes que ya existen con el dueño que su auto tiene hoy, que es el que ya mostraban: el sistema anterior no traía otro dato confiable. Por eso conviene correr el actualizador **antes** de traspasar cualquier auto. Un traspaso hecho antes de actualizar movería sus órdenes antiguas al dueño nuevo, y esa información no se puede reconstruir. La aplicación nueva sobre una base sin ese paso falla al leer órdenes: el actualizador va siempre antes.

9.2 La versión 1.2.0: exportar el inventario

Agrega el aviso de cliente repetido al crear uno nuevo y el botón **Exportar Excel** de Inventario. No cambia el esquema. Aun así, un equipo que venga de la 1.1.2 no tiene el paso `ordenes.cliente_id` de la sección 9.1, así que el orden es el de siempre: primero `actualizar.exe` y después la aplicación nueva.

La planilla la escribe `src/xlsx.py`, igual que la plantilla de carga y los reportes. El sistema no depende de `openpyxl` ni de otra biblioteca de Excel, y no conviene agregarla: el ejecutable tendría que empaquetarla y la del repositorio ya alcanza para una hoja sin estilos.

10 Problemas frecuentes

"No se pudo conectar usando DATABASE_URL"

El servicio de PostgreSQL no está corriendo, o el `.env` no está junto al ejecutable, o la contraseña cambió. Verificar el servicio en *Servicios de Windows* y probar la conexión con `psql -U postgres -d lubriexpress`.

Los diálogos salen en inglés

Falta `qtbase_es.qm` dentro de `_internal\PySide6\Qt\translations\` en la carpeta empaquetada. Rehacer el empaquetado con el `.spec` del repositorio.

Los reportes o los PDF no aparecen donde se esperaba

Se guardan en la carpeta *Documentos* **del usuario que tiene la sesión de Windows abierta**, en `Documentos\Lubri-Express`. Si hay varias cuentas en el equipo, cada una tiene la suya.

El stock no cuadra con la repisa

Se corrige contando, con **Ajustar stock** (queda registrado en el Kardex). Nunca editando la base de datos a mano: el Kardex dejaría de explicar el stock, que es justamente lo que el sistema anterior no hacía.

11 Verificación después de instalar

1. El ejecutable abre y muestra la ventana de inicio de sesión.
2. Se entra con el administrador creado por consola.
3. Los botones de los diálogos están en español.
4. Las pestañas abren sin errores: seis para el usuario normal, siete para el supervisor (suma **Finanzas**) y ocho para el administrador (suma también **Usuarios**).
5. Una venta de prueba baja el stock y aparece en el Kardex.
6. Una orden de prueba se guarda y exporta su PDF.
7. `Documentos\Lubri-Express\` se crea al exportar.
8. `respaldar.exe` deja un archivo y lo copia a OneDrive.
9. `actualizar.exe` dice que el esquema ya está al día.
10. Una patente que ya pertenece a otro cliente ofrece **traspasarla** (sección 9.1), y las órdenes anteriores del auto siguen a nombre de su dueño de entonces.

11. El número de versión de la barra de estado es el de la carpeta que se copió.
12. Con la sesión de administrador, **Finanzas** abre sin errores y **Cerrar sesión** vuelve al inicio de sesión.
13. Un gasto registrado en **Caja** aparece en el día correcto —si cayera en el día siguiente, el huso horario del servidor no está llegando a la conexión—.
14. La tarea programada aparece en el Programador de tareas.
15. Al día siguiente, hay un respaldo nuevo.